一、 问题概述

（一）问题描述分析

本实验基于规则的传统路径规划方法，如 A*、Dijkstra 等，实现在给定场景中的机器人避障寻径问题，找到从给定起点 s 到指定终点 e 的最短路径。

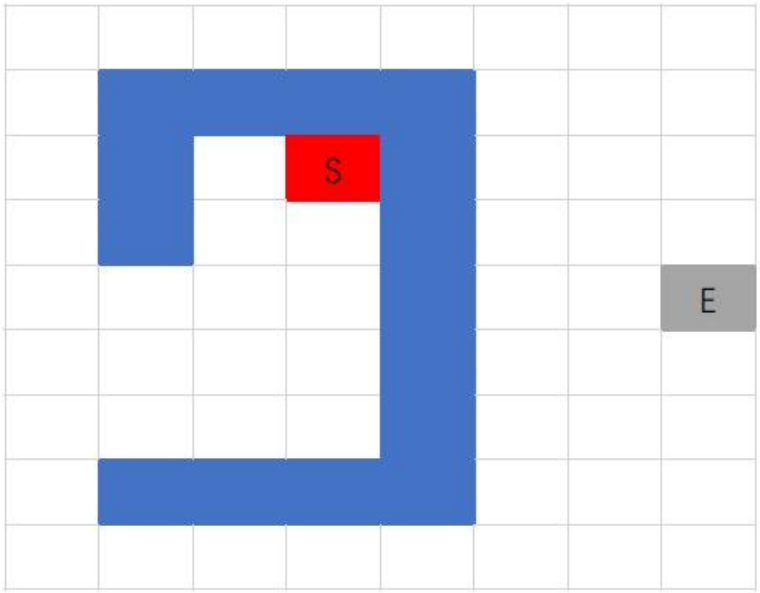


图 1：给定的实验场景图

（二）解决问题的思路和想法

基于想要实践原理课上所学知识的想法，本实验采用 **A*算法**进行最短路径的搜索。A*算法的实现关键是定义和求解启发式函数 f(x) 的值。其中 f(x)=g(x)+h(x),g(x)为从起点 start 到当前节点 current 的实际代价值,h(x)为从当前节点 current 到终点 end 的预估代价值。因此，想要通过 A*算法求解出正确的最短路径，就要定义正确的 g(x)和 h(x)函数。具体实现思路如下：

1. 初始化：将提供的场景图使用矩阵表示（左上角为基准点）。可通行区域用 0 表示，障碍物区域用 1 表示，起点 s 用 3 表示，终点 e 用 2 表示。
2. 构造 A*算法所需的启发式函数 f(x)：依据课上所学，一般可以用当前点到终点的曼哈顿距离或欧几里得距离的值来充当 h(x)的值。由于曼哈顿距离无需像欧几里得距离的求解那样反复平方开方，编写程序更简便，因此选择曼哈顿距离作为 h(x)的值。同时为了简化问题，定义当前点和起点的曼哈顿距离为 g(x)的值。

3. **制定路径规划的规则：**规定机器人只能向上、下、左、右四个方向行动，不能走斜线，不能直接穿过障碍物，不能走到障碍物上。
4. 构建搜索树以可视化图搜索过程：从起点开始，逐步扩展搜索树，探索所有可能的路径，同时记录每个节点的前驱节点和代价值。
5. 路径重建：一旦找到终点，从终点开始回溯，重建一条从起点到终点的路径。
6. 可视化搜索结果：可视化图搜索的搜索树以及最终找到的最短路径。

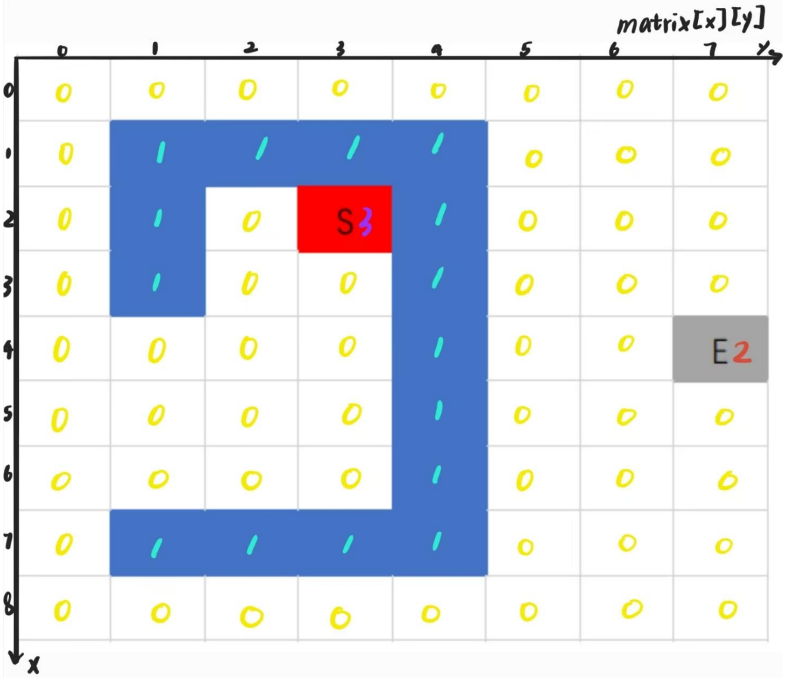


图 2：场景图的矩阵表示

二、算法设计

（一）算法原理

本次实验程序采用 A*算法来构造核心的寻径代码。A*算法是一种启发式搜索算法，通过评估每个节点的总代价来寻找最短路径。当启发函数设置得当，一定可以找到最优解。启发函数值由 g(x)和 h(x)两部分组成。本算法中的 g(x)和 h(x)的设置已在上文介绍清楚。

同时也定义了一些其他函数来可视化需要展示的结果。

（二）算法功能与设计思路

A*算法的主要功能是找到从起点到终点的最短路径，并避开障碍物。A*搜索算法

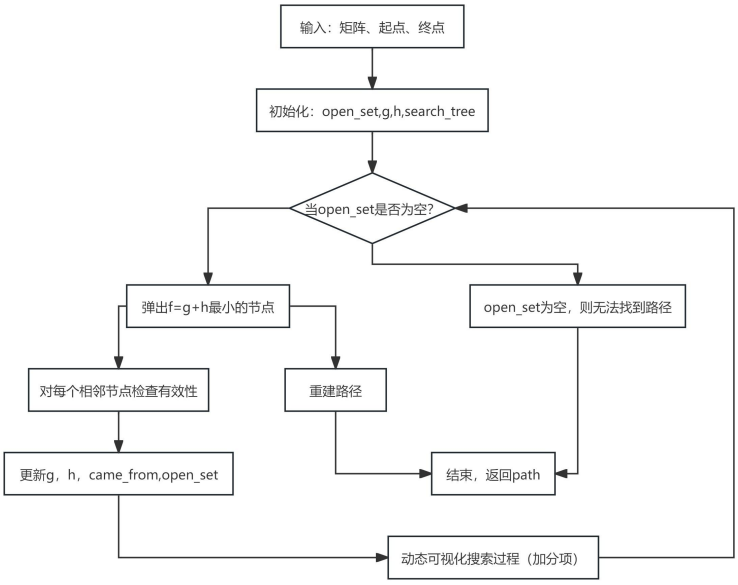
的设计思路如下：

1. 初始化地图矩阵、起点、终点和障碍物。起点坐标为（2，3），终点坐标为（4，7）

2. 借助堆来存储待探索的相邻节点和启发函数值。使用堆，是因为总能保证 pop 的堆顶元素为启发函数值最小的节点，无需另外设计排序算法维护。

3. 从起点开始扩展，每次扩展上、下、左、右且非障碍物的相邻节点，并记录每个节点的前驱节点，便于最终路径的构造。

（三）算法流程图



三、算法实现--本实验在 anaconda 的 spyder 编辑器下完成

（一）核心函数

```python
#利用曼哈顿距离来做当前节点current到end的预估代价值h(x)
def manhattan_distance(current, end):
    return abs(current[0] - end[0]) + abs(current[1] - end[1])
```

```python
#A*算法寻找最短路径
def a_star(matrix, start, end):
    rows, cols = matrix.shape
    directions = [(-1, 0), (1, 0), (0, -1), (0, 1)]  # 上下左右四个方向
    open_set = []
    heappush(open_set, (0, start))#引入堆进行维护，使得堆顶总是f=g+h最小的
    came_from = {}#记录父节点的
    g = {start: 0}
    h = {start: manhattan_distance(start, end)}

    search_tree = nx.DiGraph()  # 有向图表示搜索树
    # 初始化动态可视化
    plt.figure(figsize=(9, 8))
    cmap = plt.cm.colors.ListedColormap(["white", "blue", "gray", "red", "yellow", "lightgreen"])
    bounds = [0, 1, 2, 3, 4, 5, 6]
    norm = plt.cm.colors.BoundaryNorm(bounds, cmap.N)

    while open_set:
        _, current = heappop(open_set)

        # 如果到达终点，重建路径
        if current == end:
            path = []
            while current in came_from:
                path.append(current)
                current = came_from[current]
            path.append(start)
            return path[::-1], search_tree  # 返回路径和搜索树

        for direction in directions:
            neighbor = (current[0] + direction[0], current[1] + direction[1])
            # 检查邻居是否在矩阵范围内，且不是障碍物
            if 0 <= neighbor[0] < rows and 0 <= neighbor[1] < cols and matrix[neighbor[0], neighbor[1]] != 1:
                tentative_g= g[current] + 1

                if neighbor not in g or tentative_g < g[neighbor]:
```

```python
                    came_from[neighbor] = current
                    g[neighbor] = tentative_g
                    h[neighbor] = tentative_g + manhattan_distance(neighbor, end)
```

```python
                    search_tree.add_edge(current, neighbor)  # 在搜索树中添加边

                    # 动态更新矩阵
                    if neighbor != start and neighbor != end:
                        matrix[neighbor[0], neighbor[1]] = 5  # 用 5 表示正在搜索的节点
                    if neighbor==end:
                        break
                    # 动态可视化
                    plt.imshow(matrix, cmap=cmap, norm=norm, origin="upper", extent=[0, matrix.shape[1], 0, matrix.shape[0]])
                    plt.title("A* Search Dynamic Visualization")
                    plt.grid(True, which='both', color='gray', linewidth=0.5, linestyle='--')
                    plt.xticks(range(matrix.shape[1]))
                    plt.yticks(range(matrix.shape[0]))
                    plt.pause(0.5)  # 暂停 0.5 秒以动态显示

                    matrix[neighbor[0], neighbor[1]] = 0

    return None,search_tree  # 如果没有找到路径，返回 None 和搜索树
```

```python
def visualize_all():
    # 输入场景图矩阵
    print("输入场景图的矩阵表示（每行用空格分隔，输入空行结束,0代表可通行区域，1代表障碍区域）：")
    matrix = []
    while True:
        line = input()
        if line.strip() == "":
            break
        matrix.append(list(map(int, line.split())))

    # 输入起点和终点
    start = tuple(map(int, input("请输入起点坐标 (x,y)：").split()))
    end = tuple(map(int, input("请输入终点坐标 (x,y): ").split()))

    matrix = np.array(matrix)
    matrix[start[0], start[1]] = 3  # 用 3 -- red表示起点
    matrix[end[0], end[1]] = 2      # 用 2 -- gray表示终点

    # 动态可视化搜索过程
    path, search_tree = a_star(matrix, start, end)
    if path is None:
        print("未找到从起点到终点的路径！")
        return

    # 在矩阵中标记路径
    for point in path:
        if point != start and point != end:
            matrix[point[0], point[1]] = 4  # 用 4 表示路径

    # 最终显示路径
    plt.figure(figsize=(9, 8))
    cmap = plt.cm.colors.ListedColormap(["white", "blue", "gray", "red", "yellow", "lightgreen"])
    bounds = [0, 1, 2, 3, 4, 5, 6]
    norm = plt.cm.colors.BoundaryNorm(bounds, cmap.N)
    plt.imshow(matrix, cmap=cmap, norm=norm, origin="upper", extent=[0, matrix.shape[1], 0, matrix.shape[0]])
    plt.title("A* Pathfinding Visualization")
    plt.grid(True, which='both', color='gray', linewidth=0.5, linestyle='--')
    plt.xticks(range(matrix.shape[1]))
    plt.yticks(range(matrix.shape[0]))
    plt.show()

    # 可视化搜索树
    visualize_tree_with_matplotlib(search_tree, start, end)
```

（二）核心函数实现细节

1. manhattan_distance(current, end)

   - 计算当前点和终点的曼哈顿距离，即|x1-x2|+|y1-y2|。作为 h(x)用于 A*算法中，评估从当前节点到目标节点的预估代价。

2. a_star(matrix, start, end)

   - 获取矩阵的行数和列数
   - 定义四个可能的移动方向（上下左右）
   - 初始化堆 open_set、came_from、g、h 的值
   - 当 open_set 不为空时，循环执行：

     1. 从 open_set 取出 f 值最小的节点，如果是节点是终点，则重建路径并返回

     2. 如果不是，则检查相邻节点是否合法且不是障碍物

     3. 将未被访问过的有更小 f 值的相邻节点，加入 open_set

3. visualize_all():
   - 等待输入场景的矩阵表示、起点和终点坐标
   - 调用 a_star 函数寻找最短路径和搜索树
   - 调用 matplotlib 库的相关函数，显示最终的路径图、搜索树、动态化展示最短路径的查找过程
4. **其他可视化结果的相关函数：**
   - draw_tree_with_matplotlib(tree, root, pos=None, level=0, x=0, dx=1.0)：使用 matplotlib 绘制图搜索的过程图
   - visualize_tree_with_matplotlib(tree, start, end, max_rows_per_page=6):分页绘制搜索树（因为有 21 层 测试的时候没办法画在一张图里）

（三）核心函数的输入输出

1. manhattan_distance(current, end)
   - **输入：**当前节点坐标 current，终点坐标 end
   - **输出：**从 current 到 end 的曼哈顿距离
2. a_star(matrix, start, end)
   - **输入：**matrix 场景的 numpy 数组，0 表示可通行区域，1 表示障碍物区域；起点 start 坐标和终点 end 坐标
   - **输出：**存在的最短路径 path 和构建的搜索树 search_tree
3. visualize_all():
   - **输入：**用户输入的场景的矩阵化表示，起点、终点的坐标
   - **输出：**最终的路径、搜索树等可视化的结果

（四）数据结构定义

1. **元组：**
   - （x，y）：各个点的坐标表示
2. **字典：**
   - g：存储每个节点的实际代价

- h：存储每个节点的预估代价
- came_from：记录每个节点的前驱节点，用于路径重建
- pos：记录搜索树中每个节点的位置信息。
- levels：按层次存储搜索树的节点，用于分页绘制搜索树。

3. **数组：**
   - matrix：NumPy 数组，描述场景、起点、终点
4. **堆（优先队列）：**
   - open_set:使用 heap 的相关函数，构建一个 f 值的堆，始终维持堆顶的元素为 f 值最小的节点

四、 实验结果

（一）结果展示

根据实验要求，得到的实验结果如下：

1. 未采用 hard code 的方式，而是需要用户自行输入场景图、起点、终点：

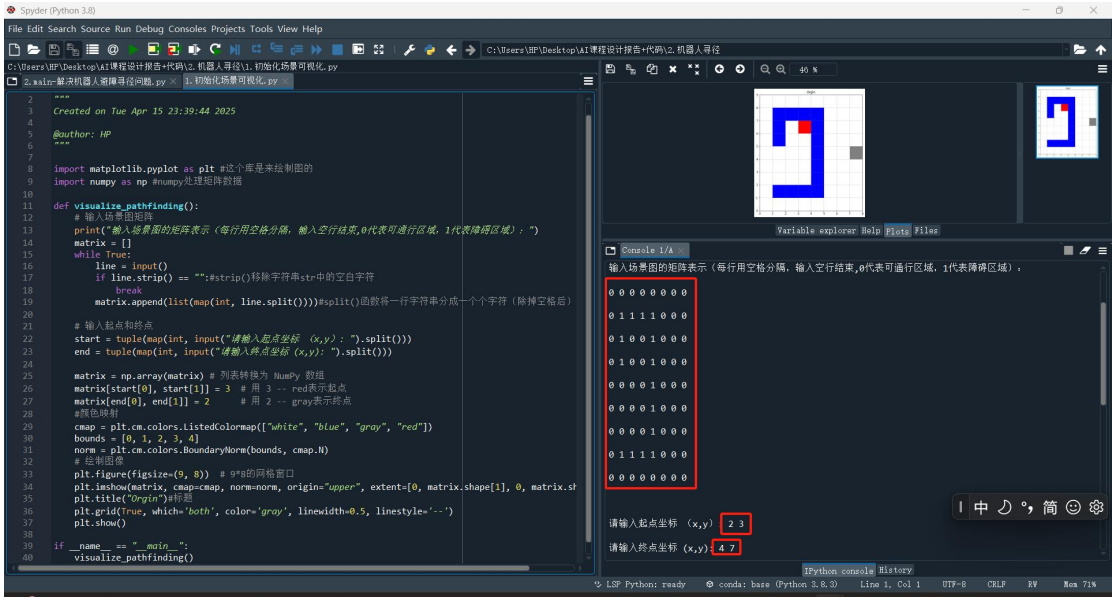


图 3：输入的场景图矩阵表示、起点、终点

2. 场景图的矩阵表示、机器人起始点和终止点的坐标：
   - 场景图的矩阵表示如图 3 所示，0 代表可通行区域，1 代表障碍物区域
   - 起始点坐标（2，3）
   - 终止点坐标（4，7）【具体的坐标系见图 2 所示】

3. 从起始点到终止点的最短路径：

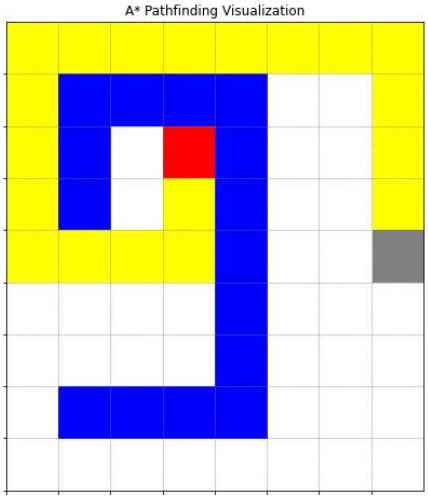


图 4：最短路径

如上图所示，黄色标出的路径即为在规定的 h（x）、g（x）值和指定规则下--即规定机器人只能向上、下、左、右四个方向行动、不能走斜线、不能直接穿过障碍物、不能走到障碍物上，得到的最短路径。

4. 可视化最短路径图搜索的过程：

得到的最短路径的图搜索过程图如下：

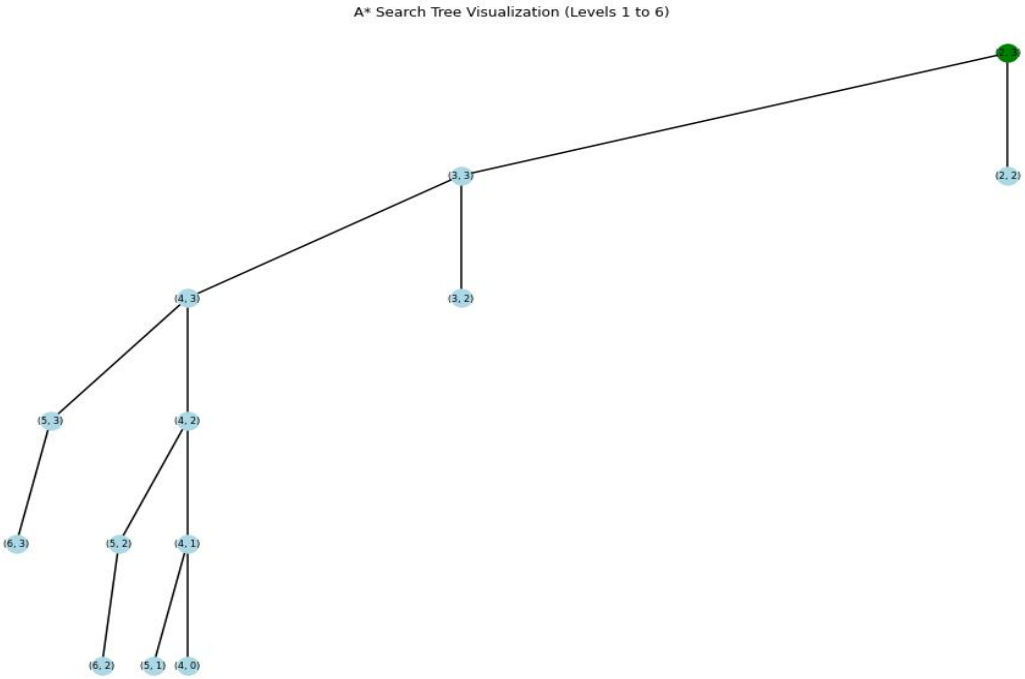


图 5：搜索树的 1-6 层

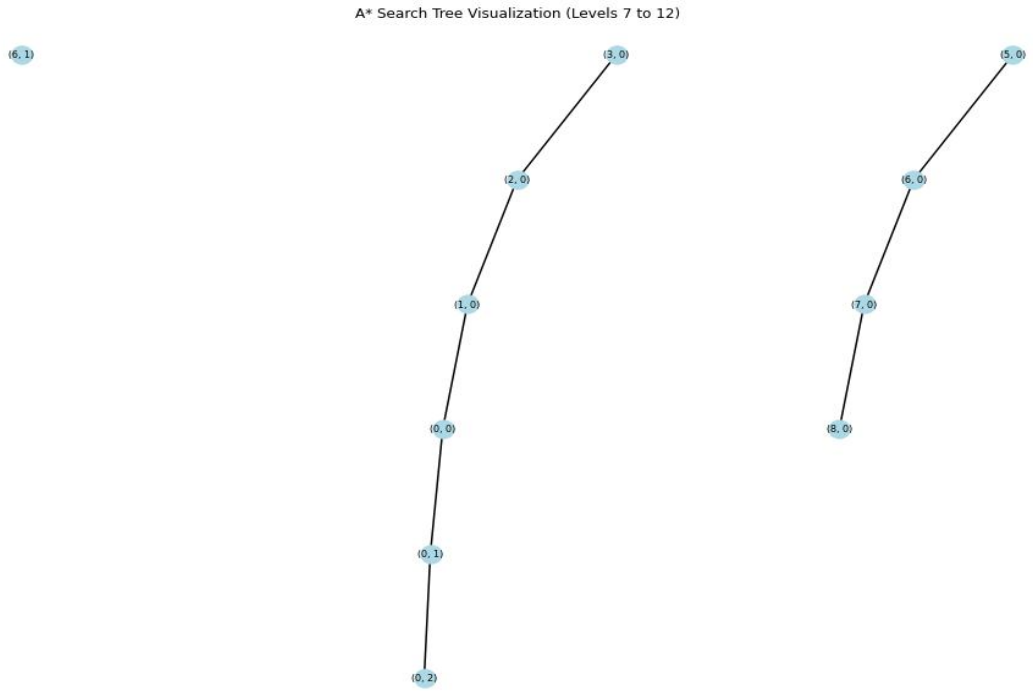


图 6：搜索树的 7-12 层

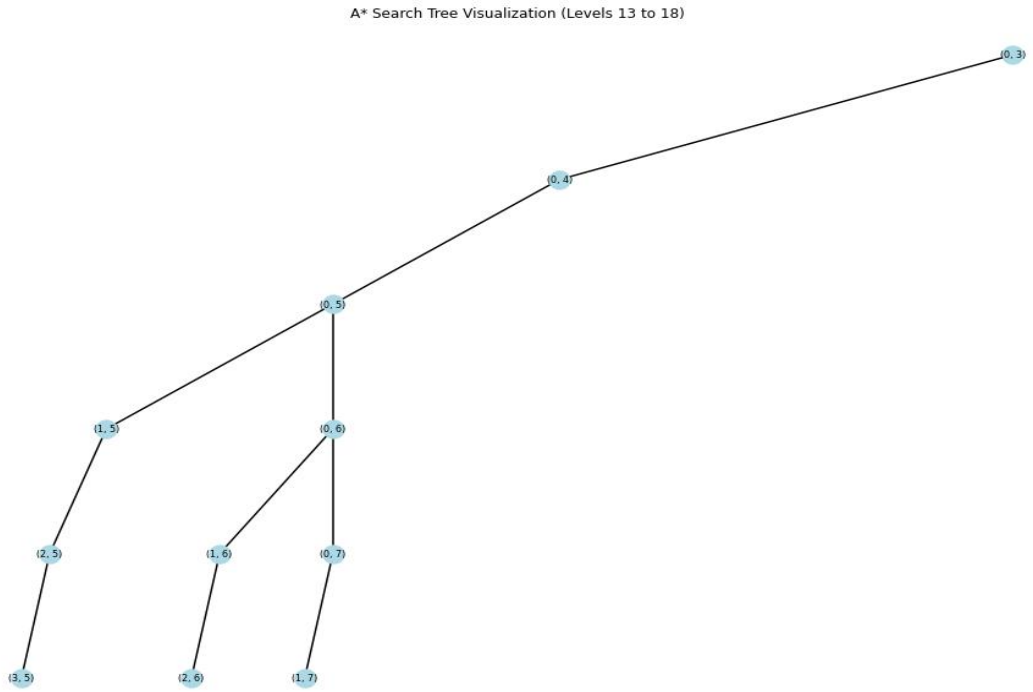


图 7：搜索树的 13-18 层

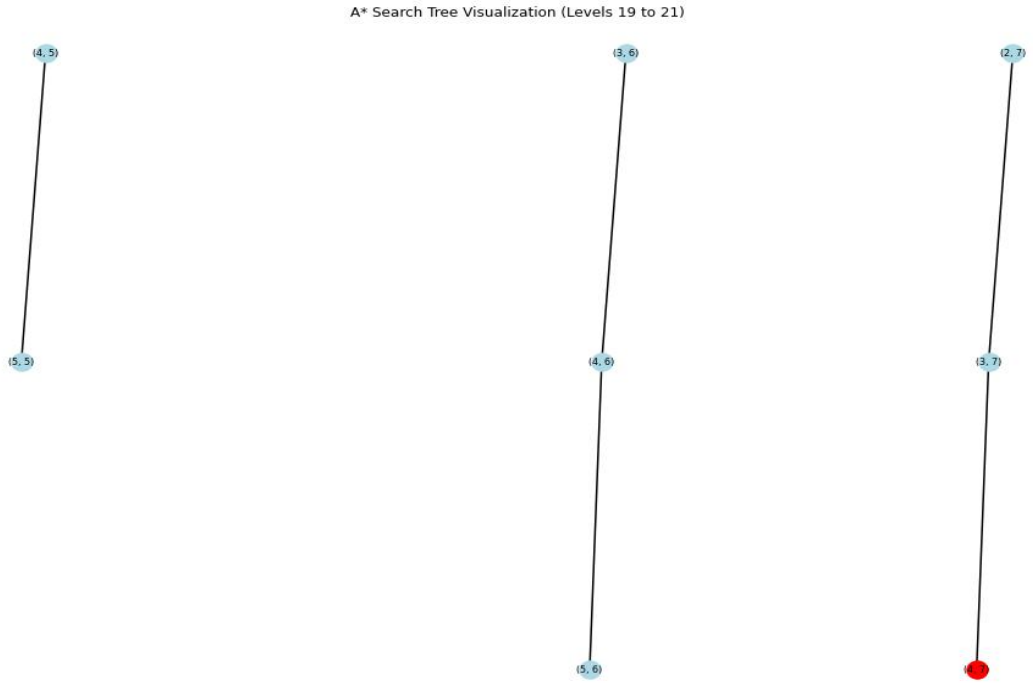


图 8：搜索树的 19-21 层

（二）结果分析

为了验证 A*算法和最短路径求解的正确性，我根据制定的规则，进行了手动验证。验证过程如下：

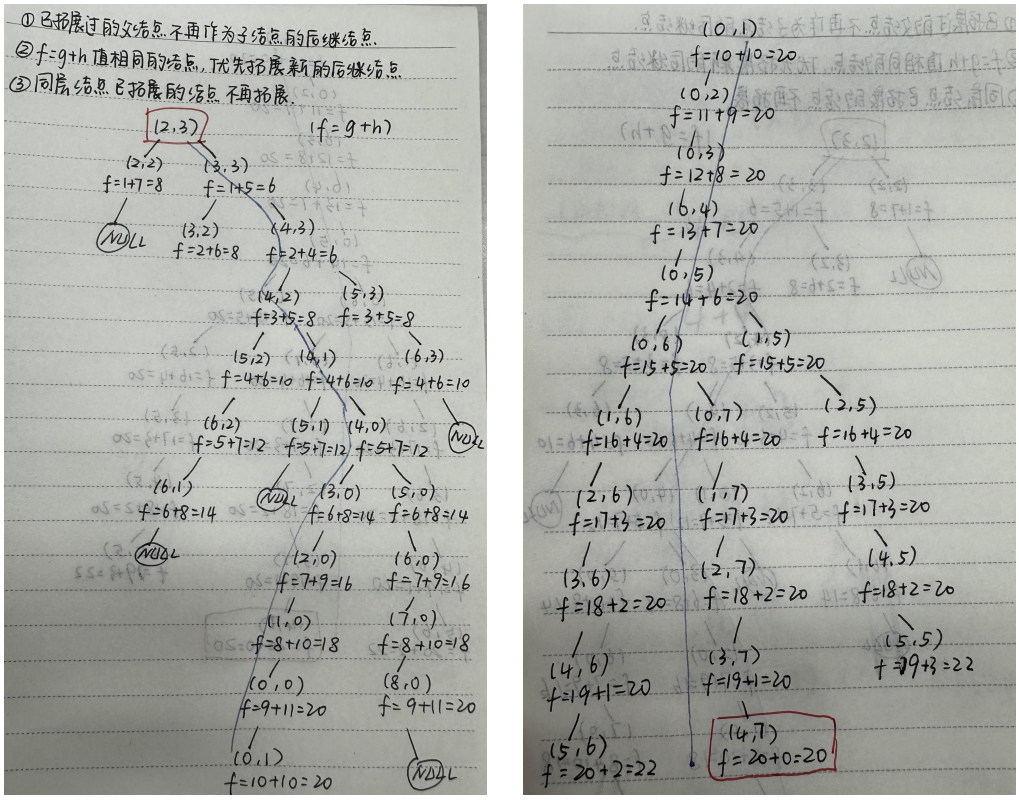


图 9：手动验证过程

最终，手动模拟出的路径为：

(2,3)-->(3,3)-->(4,3)-->(4,2)-->(4,1)-->(4,0)-->(3,0)-->(2,0)-->(1,0)-->(0,0)-->(0,1)-->(0,2)-->(0,3)-->(0,4)-->(0,5)-->(0,6)-->(0,7)-->(1,7)-->(2,7)-->(3,7)-->(4,7)

与算法得到的最短路径相同，并且构造出的搜索树也是一致的（个别子树位置不一样而已。

（三）实验过程（调试过程中遇到的问题在第六部分提及）

第一阶段：通过编写可视化初始场景的程序，了解了 python 中关于可视化的相关库 matplotlib 及其应用；

第二阶段：先不管可视化图搜索的结果，根据制定的路径规则和 g、h 函数值，编写 A*算法的核心代码，找出最短路径，并进行验证；

第三阶段：在 A*算法的基础上，完成搜索树的可视化表示。首先，通过 draw_tree_with_matplotlib 函数递归地确定搜索树中每个节点的位置。然后，使用这些位置信息，分层次地绘制搜索树，每 5 层作为一个单独的画面。

五、选作内容（加分项）

（一）分析所设计算法的局限性及改进点

1. 总代价函数 f(x)的选择：

- **预估代价函数 h（x）：**

局限性：本题目的解答选取曼哈顿距离作为预估代价函数 h（x），虽然计算简单，避免了复杂的平方开方运算，但是路径决策上可能会出现多个 f 值相同的节点（如图 9 计算所示），就需要遍历很多次，增加了时间复杂度。

改进点：因此可以选择欧几里得距离作为预估代价函数，虽然要进行平方开方计算，但是效率可能比遍历多个节点的方法更高。

- **实际代价函数 g（x）：**

局限性：为了简化题目，本题求解的时候设所有路径权重均为 1，但在实际的寻径应用场景中，不同的路径权重肯定是不尽相同的。

改进点：可以设计更复杂的场景，比如横向路径权重为 x，径向权重为 y，以更贴近真实的应用场景。

2. 路径搜索方向：

局限性：为了简化题目，本题规定只能向上、下、左、右四个方向进行搜索。

改进点：可以增加斜向搜索的选择（八方向）。

（二）最短路径查找的动态化展示

此程序运行起来后，会动态展示每一步正在搜索的节点，并用绿色标识，然后每 0.5s 截取当前移动的位置截图。动态化展示的实现其实就是在 a_star 函数寻找可能路径的过程中，增加当前查找状态的展示。

```
for direction in directions:
    neighbor = (current[0] + direction[0], current[1] + direction[1])
    # 检查邻居是否在矩阵范围内，且不是障碍物
    if 0 <= neighbor[0] < rows and 0 <= neighbor[1] < cols and matrix[neighbor[0], neighbor[1]] != 1:
        tentative_g= g[current] + 1

        if neighbor not in g or tentative_g < g[neighbor]:
            came_from[neighbor] = current
            g[neighbor] = tentative_g
            h[neighbor] = tentative_g + manhattan_distance(neighbor, end)
            heappush(open_set, (h[neighbor], neighbor))
            search_tree.add_edge(current, neighbor)  # 在搜索树中添加边

            # 动态更新矩阵
            if neighbor != start and neighbor != end:
                matrix[neighbor[0], neighbor[1]] = 5  # 用 5 表示正在搜索的节点
            if neighbor==end:
                break
            # 动态可视化
            plt.imshow(matrix, cmap=cmap, norm=norm, origin="upper", extent=[0, matrix.shape[1], 0, matrix.shape[0]])
            plt.title("A* Search Dynamic Visualization")
            plt.grid(True, which='both', color='gray', linewidth=0.5, linestyle='--')
            plt.xticks(range(matrix.shape[1]))
            plt.yticks(range(matrix.shape[0]))
            plt.pause(0.5)  # 暂停 0.5 秒以动态显示

            matrix[neighbor[0], neighbor[1]] = 0
```

图 10：动态化展示的代码

以下是动态化展示过程中每隔 0.5s 的 shot 图，展示了每一个可能的路径节点的探测顺序（可以结合上述图 5-8 的图搜索过程的可视化展示理解每一次搜索的路径）：

图 11：动态化展示搜索过程的截图

六、总结与分析

（三）问题与解决方案

Question1：第一阶段编写可视化初始场景时，将坐标系原点定在了左下角，输入起始点、终止点坐标后绘制出的场景图是错误的。

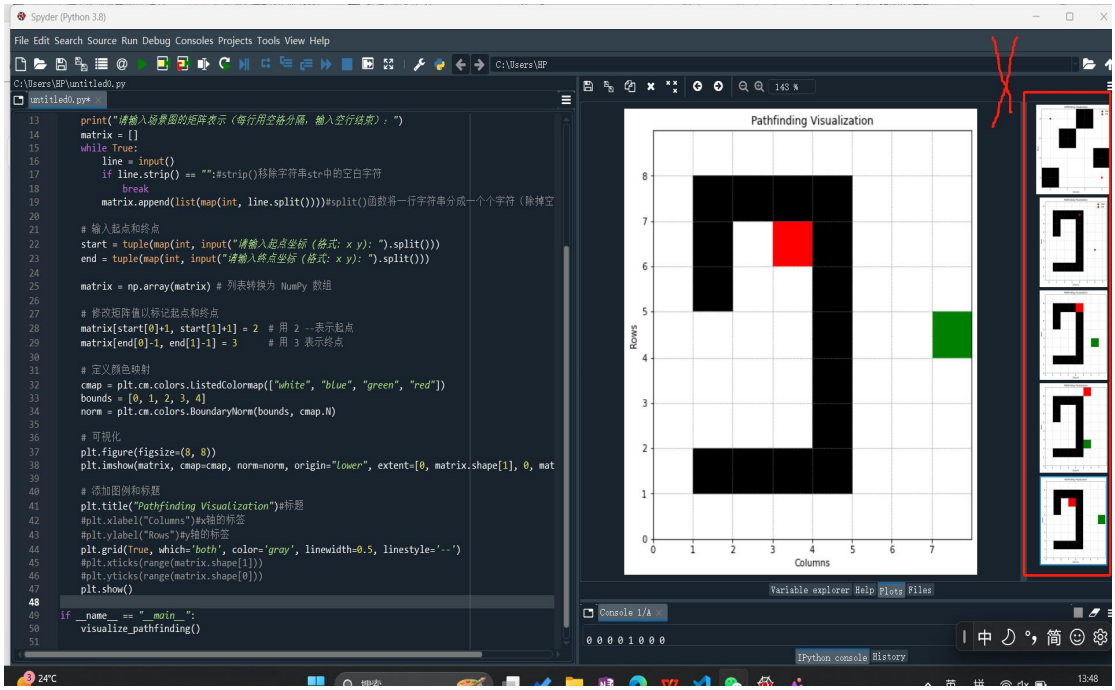


图 12：绘制错误展示

Solution1：虽然最后通过一些转换，可以在该坐标系下正确地展示起始点、终止点，但是后续发现**将原点定在左上角**，无论在场景的初始化还是在后续的最短路径查找过程的动态化展示，函数逻辑都更简单，更容易实现。

Question2：在第三阶段绘制图搜索过程时，一开始是采用自带的 spring_layout 弹簧布局函数来绘制图，但是特别拥挤，不美观，更换了很多种自带的布局函数，但效果均不佳。

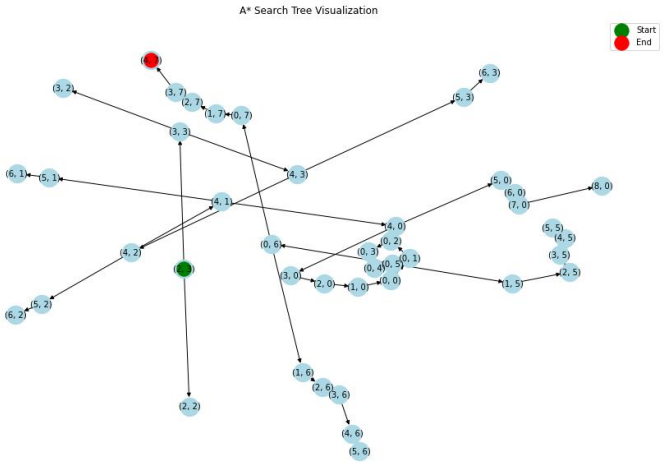


图 13：spring_layout 函数绘制图展示

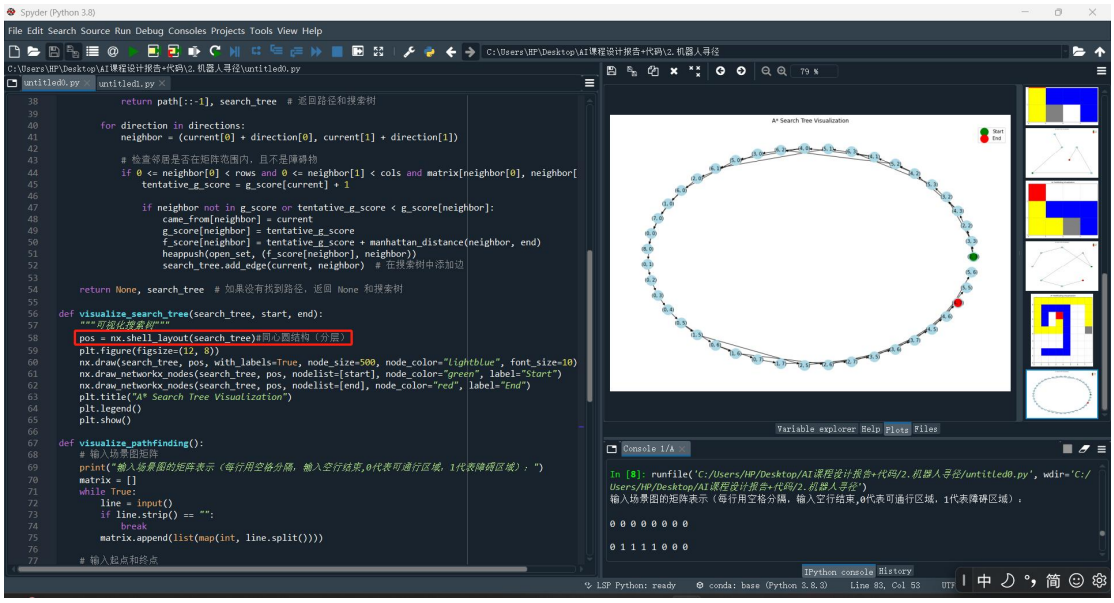


图 14：shell_layout 函数绘制图展示

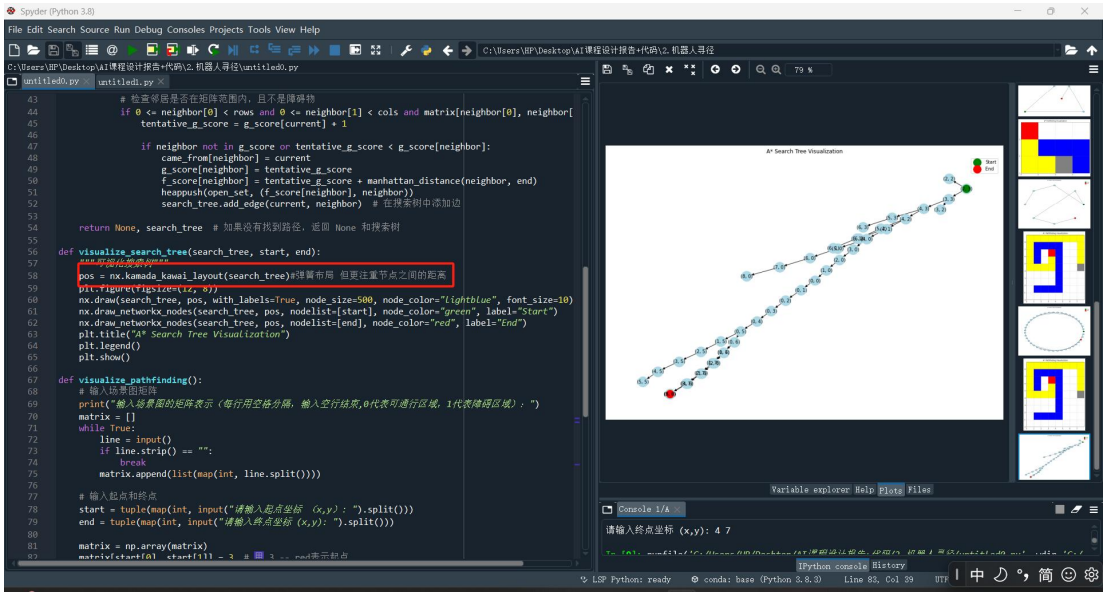


图 15：kamada_kawai_layout 函数绘制图展示

Solution2：最后决定不采用自带的布局函数，自己手动绘制。然后就去搜了一些参考资料，手动递归地绘制出搜索树。由于 21 层在一张图片中不可避免地会相互遮挡覆盖，于是每张图片只绘制 5 层深度的搜索树。

Question3：在完成选作内容时，想动态展示最短路径的搜索过程，但是回溯的路径一直不对，就像图 14 所示，搜索过的点全部变成绿色，无法分辨什么时候在搜索哪个节点。

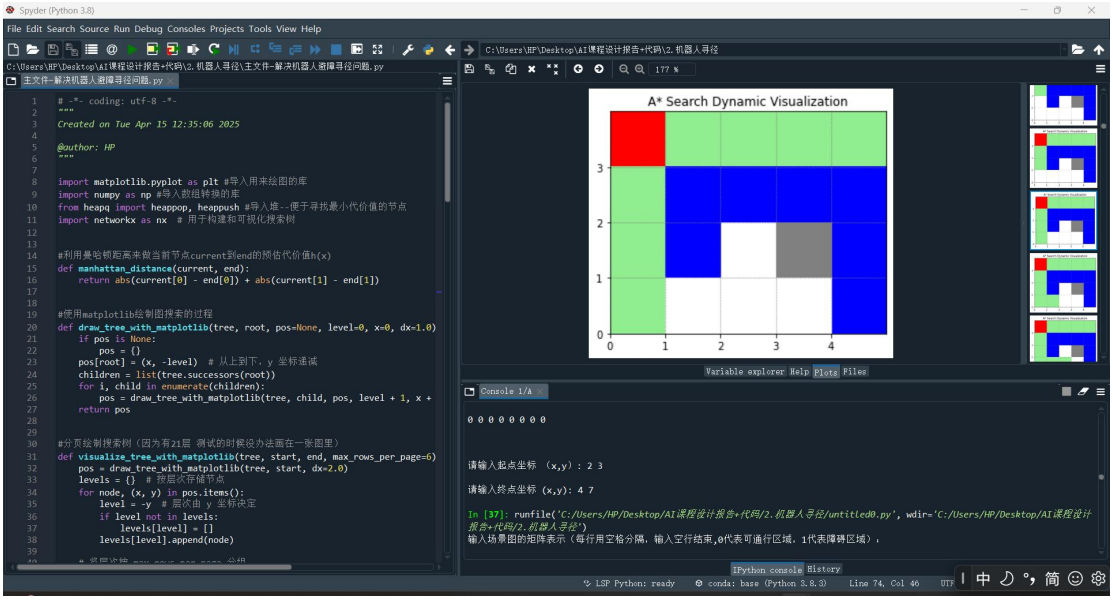


图 16：回溯错误展示

Solution3：最后选择只动态展示当前搜索的节点，而不是拖着所有搜索过的节点。正好搜索的节点动态展示的顺序就是图搜索树搜索的顺序，可以很好地理解图搜索过程的搜索过程。

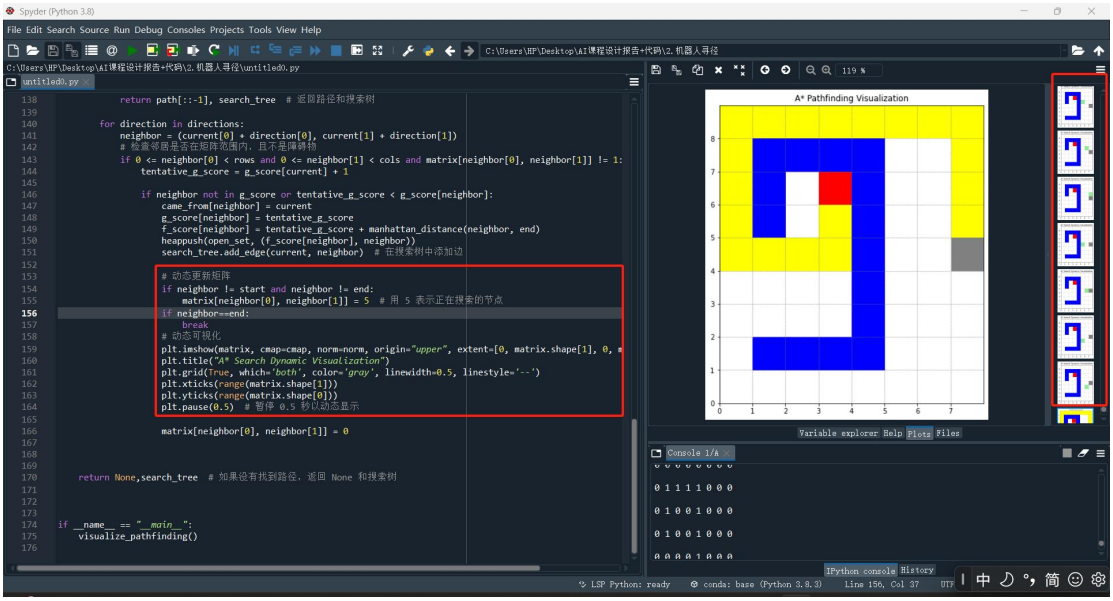


图 17：修改后的动态展示过程

（四）收获与思考

1. 对 A*算法的思考：

之前对 A*算法的理解还只是停留在理论层面，通过此次实验作业，加深了我对 A*算法的理解。通过查阅一些资料，也认识到了 A*算法的完备性以及启发函数选择的重要性。同时提到的 Dijskra 算法，其实上学期也有写过相关的 c++算法，我个人觉得 A*算法比 Dijskra 算法更容易理解，实践起来也更加容易。

2. 调试优化代码能力的提高：

我其实本身写 c++的代码比较多，对 python 的很多操作还不是很熟悉，编写程序的过程中有点类似于先写出 c++，再通过资料辅助转成 python 的感觉。但是我也在这个过程中认识到了 python 的强大和简便性。在 c++中如果要写一个优先队列，肯定避免不了每次加入节点时都进行排序，这就又要多谢一个函数，就使得程序的逻辑很复杂。而 python 直接引入 heap 库及其相关函数就可以使得堆顶始终为最小值的节点，非常方便。然后之前在高程课上实现过动态化的展示（彩球大作业），但是当时就用了很多老师提供的可视化函数，才能在 cmd 界面展示出来，就很麻烦，但是 python 导入一个绘图库就行，真的非常方便。但我觉得 python 靠缩进划分代码块还是非常不方便的，我常常

修改的时候不知道现在这句属于哪个循环哪个 if 内了。

（五）算法评价

其实在第五部分选作内容的分析里，已经提到过关于这个算法的评价。优点很明显，当前启发函数下 A*算法一定能找到最短路径，但缺点就是启发函数的选择要非常谨慎。然后就是本题目其实是非常简化版的寻径问题，要应用于真实的场景还需要修改优化的地方很多。

七、参考文献（资料）

- https://blog.csdn.net/universsky2015/article/details/137326053
- https://blog.csdn.net/Zhouzi_heng/article/details/115035298
- https://zhuanlan.zhihu.com/p/13185307595
- https://zhuanlan.zhihu.com/p/338414118